```
tells him how many ways we can initialize the object properties :
-------------------------------------------------------------------------
1) By using Object reference
2) By using Method without parameter
3) By using Method with parameter
4) By using Parameterized Constructor
5) Using the Argument Constructor
6) Using parameterized Constructor

Program on parameterized constructor with Data Validation :
-------------------------------------------------------------------------
package com.kli;

public class Product
{
  private int id;
  private String name;
  private double price;

  public Product(int id, String name, double price)
  {
  if(id <= || name==null ||name.isBlank() || price <0)
  {
    System.err.println("Invalid Data");
    System.exit(0);
  }

  this.id = id;
  this.name = name;
  this.price = price;

  }

  @Override
  public String toString();
  {
  return "Product [id=" + id + ", name=" + name + ", price=" + price + "]";
  }

}

package com.kli;

import com.kli.Product;

public class ProductTest
{
  public static void main(String[] args)
  {
      Product p1 = new Product(100,"laptop", 90000);
      System.out.println(p1);
  }
}
-------------------------------------------------------------------------
How to write setter and getter for our non static fields :
-------------------------------------------------------------------------
public class Customer
{
  private double customerBill;   //Non static field

  //Initializing the non static field                    //Parameterized constructor
  public Customer(double customerBill)                   Customer c1 = new Customer(1000);
  {
    this.customerBill = customerBill;
  }

  //To modify the existing object data we should setter   c1.setCustomerBill(1500); //Setter
  public void setCustomerBill(double customerBill)
  {
    this.customerBill = customerBill;
  }


  //To read the private non static field value we should use getter.
  public double getCustomerBill()                        c1.getCustomerBill();
  {
    return this.customerBill;
  }
}
-------------------------------------------------------------------------
Important points regarding setter and getter :
-------------------------------------------------------------------------
1) Setter and getter we can write only for non static fields.

2) Setter is mainly used to modify the existing object data, one
   data at a time.

3) Setter method return type is always void and it will modify
   only one non static field value in the current object.

4) Getter is mainly used to read the private non static field
   value from the current object. Getter method return type is always
   non-void (Depends upon non static field data type)

FINAL CONCLUSION :
-------------------------------------------------------------------------
Parameterized Constructor : To initialize the Object properties (non static Field) with user values.

Setter : To modify the existing object data (Only one data at a time) OR Writing Operation

Getter : To read/retrieve private data value outside of BLC class. (Reading Operation)

toString() : To print Object properties (Non static Field)
-------------------------------------------------------------------------
**** What is Encapsulation ?
(Accessing our private data via public methods like setter and getter is encapsulation)
```


Capsul in Medical term

Java Encapsulation

```
Binding the private non static field with its associated non static method in a single unit is called
Encapsulation.

Encapsulation ensures that our private data (Object Properties) must be accessible via public
methods like setter and getter.

It provides security because our data is private (Data Hiding) and it is only accessible via public
methods WITH PROPER DATA VALIDATION.

In java, class is the example of encapsulation.

How to achieve encapsulation in a class :

In order to achieve encapsulation we should follow the following two
techniques :

1) Declare all the data members with private access modifiers (Data Hiding OR Data Security)

2) Write public methods to perform read(getter) and write(setter) operation on these private data
like setter and getter.

Note : If we declare all our data with private access modifier then it is called TOTALLY
ENCAPSULATED CLASS. On the other hand if we declare our data other than private access
modifier then it is called Linearly Encapsulated class.

//Program on Encapsulation :
-------------------------------------------------------------------------
package com.rev1.encapsulation;

public class Employee
{
  private String name;
  private double salary;

  public Employee(String name, double salary)
  {
  super();
  this.name = name;
  this.salary = salary;
  }

  @Override
  public String toString()
  {
  return "Employee [name=" + name + ", salary=" + salary + "]";
  }

  //Getter
  public String getName()
  {
  return name;
  }

  //Setter
  public void setName(String name)
  {
  this.name = name;
  }

  //Getter
  public double getSalary()
  {
  return salary;
  }

  //Setter
  public void setSalary(double salary)
  {
    if(salary < this.salary)
    {
      System.err.println("Invalid Data");
      System.exit(0);
    }
  this.salary = salary;
  }

}


package com.rev1.encapsulation;

import java.util.Scanner;

public class EmployeeTest
{
  public static void main(String[] args)
  {
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter Employee Name :");
      String name = sc.nextLine();

      System.out.print("Enter Employee Salary :");
      double salary = Double.parseDouble(sc.nextLine());

      Employee emp = new Employee(name, salary);
      System.out.print(emp);

      System.out.print("Enter your Increment amount :");
      double incrementAmount = Double.parseDouble(sc.nextLine());

      emp.setSalary(emp.getSalary() + incrementAmount);
      System.out.print(emp);
      sc.close();
  }
}
-------------------------------------------------------------------------
```